PYTHON PROGRAMMING LAB MANUAL

Compilation of Lab Exercises 5, 6, and 7

NAME: Sriram Satvik **USN:** 25BTRAD049

COURSE: Python Programming (Lab) **ACADEMIC YEAR: 2026-2027**

Lab Exercise 5: Number Guessing Game

OBJECTIVE / QUESTION

Write a Python program to implement a Number Guessing Game. The program should select a secret random number between 1 and 100, then prompt the user to guess the number. For each guess, the program must provide feedback stating whether the guess was 'Too high', 'Too low', or 'Correct'. Repeat the loop until the user guesses correctly and then output the total attempts taken.

Algorithm:

1. Import the `random` module.
2. Generate a random secret integer between 1 and 100 using `random.randint(1, 100)`.
3. Initialize an `attempts` counter to zero.
4. Execute an infinite `while` loop to repeatedly read player guesses.
5. Include validation using a `try-except` block to gracefully handle non-integer inputs.
6. Increment the attempt counter with each valid guess.
7. Compare the guess with the secret number, printing 'Too high! Try again.' or 'Too low! Try again.' accordingly.
8. If the guess is correct, print a congratulations message with the total number of attempts and exit the loop.

Python Program Source Code:

```
import random

def number_guessing_game():
    secret_number = random.randint(1, 100)
    attempts = 0
    print("I have selected a secret number between 1 and 100. Can you guess it?")

    while True:
        try:
            # Take input and validate it
            guess = int(input("Enter your guess: "))
            attempts += 1

            if guess < secret_number:
                print("Too low! Try again.")
            elif guess > secret_number:
                print("Too high! Try again.")
            else:
                print(f"Congratulations! You guessed the number {secret_number} in {attempts}
attempts.")
                break
        except ValueError:
            print("Invalid input. Please enter a valid integer.")

if __name__ == "__main__":
    number_guessing_game()
```

```
SYSTEM OUTPUT:
I have selected a secret number between 1 and 100. Can you guess it?
Enter your guess: 50
Too high! Try again.
Enter your guess: 25
Too low! Try again.
Enter your guess: 42
Congratulations! You guessed the number 42 in 3 attempts.
```

Lab Exercise 6: Prime Number Check and Factorial Program

OBJECTIVE / QUESTION

Write Python programs to:
1. Check whether a given positive integer is a Prime Number or not.
2. Calculate the Factorial of a non-negative integer using an iterative loop.

Algorithms:
A. Prime Check: A prime number has exactly two distinct positive divisors: 1 and itself. To verify primality efficiently, test divisibility starting from 2 up to the square root of the number ($i \le \sqrt{N}$). If a divisor is

found, the number is composite; otherwise, it is prime.
B. Factorial Check: The factorial of n ($n!$) is computed iteratively. Start with a result variable set to 1, then loop from 1 to n, updating `result *= i` at each step. Handle negative numbers as undefined.

Python Program Source Code:

```
# Part A: Prime Number Checker
def is_prime(num):
    if num <= 1:
        return False
    # Check for factors from 2 up to sqrt(num)
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True

# Part B: Iterative Factorial Calculator
def factorial(n):
    if n < 0:
        return "Undefined for negative numbers"
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

# Execution and verification
if __name__ == "__main__":
    print("--- Part A: Prime Checker ---")
    test_numbers = [17, 12]
    for num in test_numbers:
        if is_prime(num):
            print(f"{num} is a prime number.")
        else:
            print(f"{num} is not a prime number.")

    print("\n--- Part B: Factorial ---")
    num_fact = 5
    print(f"Factorial of {num_fact} is: {factorial(num_fact)}")
```

```
SYSTEM OUTPUT:
--- Part A: Prime Checker ---
17 is a prime number.
12 is not a prime number.

--- Part B: Factorial ---
Factorial of 5 is: 120
```

Lab Exercise 7: Fibonacci Series Using Recursion

OBJECTIVE / QUESTION

Write a Python program to generate and print the Fibonacci sequence up to a specified number of terms (n) using a recursive function. Ensure that the program validates the number of terms input by the user.

Algorithm:

1. Define a recursive function `recur_fibo(n)` to find the n-th Fibonacci term.
2. *Base Case:* If $n \le 1$, return n (where $F(0)=0$, $F(1)=1$).
3. *Recursive Step:* For $n > 1$, return `recur_fibo(n-1) + recur_fibo(n-2)`.
4. Read the target number of terms (n) from the user and ensure it is positive.
5. Execute a loop from 0 to $n-1$, invoking `recur_fibo(i)` for each index, and print the terms sequentially.

Python Program Source Code:

```
# Recursive function to return the nth Fibonacci number
def recur_fibo(n):
    if n <= 1:
        return n
    else:
        return recur_fibo(n-1) + recur_fibo(n-2)

def generate_fibonacci_series(terms):
    if terms <= 0:
        print("Please enter a positive integer.")
    else:
        print(f"Generating Fibonacci series for {terms} terms:")
        fib_series = []
        for i in range(terms):
            fib_series.append(str(recur_fibo(i)))
        print(", ".join(fib_series))

if __name__ == "__main__":
    n_terms = 10
    generate_fibonacci_series(n_terms)
```

```
SYSTEM OUTPUT:
Generating Fibonacci series for 10 terms:
0, 1, 1, 2, 3, 5, 8, 13, 21, 34
```